

YAFL

Yet Another f... Language

Fabian Becker

January 14, 2026

Abstract

This report details the design philosophy and implementation of the YAFL programming language. Submitted as the final study project for the Compiler Construction course, YAFL is a statically typed, general-purpose procedural language compiling to bytecode for a modified version of the provided VM3 virtual machine.

1 General Overview

YAFL stands for "Yet another f... language". The "f..." can be replaced based on the programmer's current stance towards the language: *Fabulous* when it compiles, *Fun* when writing algorithms, or *Frustrating* when debugging.

1.1 Design Philosophy

The core idea behind YAFL is to merge the readability and expressiveness of modern languages with the structural discipline of traditional more languages to provide readable program with clear, foreseeable behavior at runtime. While the function signatures can be considered quite similar to Go, it also borrows high-level concepts like generalized `for` loops and `range` iterables from Python, making common tasks concise. However, it retains the explicit structure of C-like languages, using braces `{}` for blocks and semicolons `;` for statement termination which is familiar to almost every programmer as well as a static type system to catch issues at compile time and not at runtime.

2 Syntax and Conventions

YAFL introduces several unique syntactic elements that distinguish it from standard C-family languages.

2.1 Identifiers and Variables

Variables are visually distinct, enclosed in pipe characters (e.g., `|my var|`). This design choice immediately separates variable references from keywords and function names. Moreover YAFL places almost no restrictions on these Identifiers, only pipes and newlines characters are prohibited allowing for more expressive variable names.

2.2 Numbers

Various way to express numbers are supported in YAFL, besides the standard of writing decimal numbers. You can prefix your number with a hashtag `#` to denotes Hexadecimal numbers, moreover you can prefix any base from 2-16 before the hashtag an the number behind is interpreted in the base.

This also works for floating point numbers, here both the parts before and after the decimal point are interpreted in this base. Only the optional exponent expressed by the caret (`^`) needs to be a decimal.

In order to grasp large numbers better they can be formatted with the underscore, this formatter is ignored during parsing and works for both integers and floating point numbers.

```
1 int |hex1| <- #FF; ~~ 255
2 int |hex2| <- 16#A0; ~~ 160
3 int |dozen| <- 12#A0; ~~ 10 * 12 + 0 = 120
4
5 float |hex_fl| <- 16#F.8; ~~ 15 + 8/16 = 15.5
6 float |exp_fl| <- 10#1.5^2; ~~ 1.5 * 10^2 = 150.0
7
8 int |large number| <- 10#1_000_000;
```

2.3 Assignments

The assignment operator is `<-`, emphasizing the flow of data into the variable. Combined arithmetic operations (Addition, Subtraction, Multiplication, Division & Modulo) and assignments are also supported for variables. Simple increment and decrement operators are also implemented with the familiar `|my var|++` and `|my var|--`, however these can be only applied after the Identifiers and execute the operation and return the updated value.

```
1 int |count| <- 0;
2
3 ~~ |count| <- |count| + 1 can be expressed as
4 |count| <-+ 1;
5 |count|++;
```

2.4 Strings

String literals are delimited by double chevrons (`>>...<<`). This avoids common escaping issues found with single or double quotes and allows for easy embedding of other quote characters. The caret (`^n`) serves as the escape characters and arbitrary characters can be inserted by their hexcode (`^xAA`).

```
1 str |msg| <- >>Hello,^"World"!^n<<;
```

2.5 Comments

YAFL supports both line and block comments:

- **Line comments:** Start with `~~` and extend to the end of the line.
- **Block comments:** Enclosed within `{- ... -}`.

2.6 Function Signatures

Function declarations explicitly separate parameter types and names, and use `->` to indicate the return type.

```
1 fn calculate(int : |x|, int : |y|) -> int { ... }
```

In resemblance to assembly programs a valid YAFL program needs a function named `start()` with no parameters. The return type can be either `int` or `none`, however in both cases nothing is returned.

```
1 fn start() -> <int|none> { ... }
```

3 Language Features

3.1 Type System

YAFL is statically typed, ensuring type safety at compile time. It supports a variety of primitive and composite types.

3.2 Supported Types

- **Primitives:** `int` (signed integer), `float` (floating point with double precision), `bool` (`Yes/No`), and `str` (string).
- **Collections:** `arr` (typed arrays) and `range` (arithmetic sequences).
- **Special:** `none` (void) only used to indicate empty returns.

Primitive types are passed by reference, while Collections are passed by values. `arr` is a static array which allows for composite types like `arr'arr'str`.

3.3 Zero Initialization

Types are automatically set to their corresponding zero value if there is no assignment during the first definition. For `str` this is the empty string and for `bool` this is `No`. For arrays the corresponding primitive are used.

3.4 Input Arguments

Input arguments can be accessed by calling the `args()` function. It returns a string array with the input arguments, however it doesn't start with the program name but rather the first other argument.

3.5 Control Structures

3.5.1 If-elif-else Statements

If-statements are also implemented in the common way, were to condition needs to be of type `bool`. Sequential comparisons can be expressed via the `elif` keywords, but essentially these statements are converted to nested if-else during AST generation.

```
1 if |i| % 15 = 0 { ... }
2 elif |i| % 3 = 0 { ... }
3 else { ... }
```

3.5.2 For Loops and Iterables

The `for` loop is a generalized iterator. It abstracts over different data sources using the syntax `for <type> |var| in iterator`.

Ranges and Arrays: The built-in `range()` function generates arithmetic sequences. Uniquely, YAFL supports iteration over array literals and strings directly.

The `range()` is a special lazy iterator as it does not create the array on the stack which would result in huge memory consumption for large iterations. Rather the `range()` is represented as an array of following structure on the stack `["__RANGE__", <current>, <stop>, <step>]`. Current is initially set to the start value. The OPCODE `MKRANGE` creates this special array on stack from provided start, stop and step values, this syntax can't be recreated by user, as arrays need to have a common type. Ranges can only be of type `int`, and support negative start and stop values to begin iteration from the high

Arrays and strings also get wrapped in an iterator container of the structure `[<object>, <current>, <len>]` by the OPCODE `MKITER`.

The iteration process is then performed by the OPCODE `ITER_NEXT`, here either the next value of the iterator is pushed and a one is pushed to the stack and the container is updated or the container gets popped and a zero is pushed to the stack to indicate the end.

```
1 ~~ Range with negative step (decimal notation)
2 for int |i| in range(10, 0, -2) { ... }
3
4 ~~ Iterating over an array literal
```

```

5 | for int |x| in [10, 20, 30] { ... }
6
7 | ~ Iterating over a string
8 | for str |x| in >>Yafl_is_fun!<< { ... }

```

3.5.3 Match Statements

The `match` statement replaces complex `if-else` chains. It supports integers and strings, implicit fallthrough for empty cases, and a default `otherwise` branch.

```

1 | match |val| {
2 |     case 1 -> { print(>>One<<); }
3 |     case 2 ->
4 |     case 3 -> { print(>>Two_or_Three<<); }
5 |     otherwise -> { print(>>Other<<); }
6 | }

```

3.6 Functions

3.6.1 Forward References

3.6.2 Overloading and Default Arguments

YAFL supports function overloading, allowing multiple definitions for the same name with different parameter lists. It also supports default values for trailing arguments, simplifying API usage.

```

1 | fn log(str : |msg|, int : |level| <- 1) -> none { ... }

```

3.6.3 Builtin Functions

4 Optimizations

5 Toolchain

A Project Structure

B VM3 Modifications

To support YAFL's features, the standard VM3 architecture was extended with custom opcodes and native functions.

B.1 Custom Number Syntax

The lexer and VM support explicit base notation for integers and floats, ranging from base 2 to 16.

- **Binary:** 2#1010 (Decimal 10)
- **Hexadecimal:** 16#FF or #FF
- **Scientific Base:** 10#1.5^-2

B.2 Extended Instruction Set

New opcodes were added to handle high-level operations efficiently:

- **MKRANGE:** Creates a range object on the stack from start, stop, and step values.
- **SLEEPMS:** Pauses execution for a specified duration.

- **LEN, COPY, SLICE:** Native handling of array/string operations.
- **CAST, TYPEOF:** Support for runtime type checking and conversion.

C Demonstration Programs

The capabilities of YAFL are demonstrated through four included programs:

1. **Game of Life:** Visualizes cellular automata using arrays and loops.
2. **Run Length Decoder:** Recursively decodes strings (e.g., `3[ab]`).
3. **Black Jack:** A card game simulation using random number generation and user input.
4. **Language Demo:** Highlights overloading, slicing, and pattern matching.

D Conclusion

YAFL successfully implements a functional compiler for a non-trivial language. By incorporating features like overloading, pattern matching, and a robust type system, it exceeds basic requirements, offering a flexible toolset for general-purpose programming on the VM3 architecture.